INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES ROUEN NORMANDIE

Département GM

Projet JAVA

Simulation de trajectoire autonome d'un satellite

Encadrant:
M. A. OSMANI

Présenté par:
Mouad SHERADJ DRISSI
Amr ZAKI SALIH

Année universitaire: 2025-2026

Table des matières

1 Introduction 2
2 Synthèse des besoins 3
3 Modèle choisi 4
3.1 Modèle Physique 4
3.2 Propulsion 10
3.3 Modèle algorithmique 11
4 Code 12
4.1 Collision 26
4.2 ForceGravitationnelle 27
4.3 RéseauSatellites - Système de Communication 27
4.4 Antenne 29
4.5 Interface graphique 29
5 Contributions 30
6 Analyse des réalisations 30
7 Conclusion 31

1 Introduction

Ce projet a pour objectif de concevoir et développer un programme complet en langage Java intégrant trois dimensions essentielles : la communication réseau, une interface graphique utilisateur et une gestion de base de données.

Le système doit modéliser un environnement spatial en 2D composé de plusieurs astres, de satellites et de contraintes physiques (obstacles, météorites, ...). L'utilisateur aura la possibilité de spécifier un point de départ et une destination, et le programme calculera le parcours idéal à suivre en prenant en considération ces paramètres. Des données recueillies par des antennes seront partagées entre les satellites, créant un réseau coopératif et interconnecté.

Contexte

Ce projet s'inscrit dans le cadre de la Programmation Orientée Objet Avancée, qui a pour objectif de mettre en pratique les principes de conception et de développement vus en cours. Le thème retenu s'inspire d'un projet semestriel plus large portant sur l'analyse de données satellitaires, et en particulier sur l'étude du lien entre assèchement du sol et hauteur du sol en Europe. Pour adapter ce sujet à un contexte applicatif orienté objet, le choix de développer une simulation centrée sur les interactions entre satellites et entre satellites et utilisateurs a été réalisé. Le système devra permettre de déterminer la trajectoire optimale d'un satellite pour atteindre une position cible, tout en collectant des informations issues des autres satellites présents dans l'environnement simulé. Ce projet vise donc à concilier l'apprentissage des concepts de la Programmation Orienté Objet, en particulier en Java, avec une problématique concrète inspirée du domaine spatial.

Enjeu

L'enjeu de ce projet de simulation vise dans un premier temps à mettre en œuvre de manière concrète les concepts de programmation orienté objet tels que la communication réseau, les interfaces graphiques utilisateur ainsi que la modularité, l'héritage et l'encapsulation dans le développement d'une application concrète. Ce projet inclut donc des notions de programmation orientée objet étudiées lors de ce semestre ainsi que du semestre dernier. L'enjeu porte aussi sur le défi d'algorithmie et de simulation qu'impose un tel projet puisqu'il s'agit de déterminer des trajectoires optimales pour les satellites tout en intégrant des échanges d'informations entre ces derniers. Il s'inspire ainsi de problématiques considérées comme réelles rencontrées dans le domaine spatial, notamment en matière de communication. La réussite du projet démontrera la capacité de la programmation orientée objet à répondre à des problématiques complexes.

Besoins

Besoins fonctionnels

Le programme doit modéliser un espace composé de plusieurs astres (planètes, lunes, ...), satellites, et contraintes physiques telles que les obstacles (débris spatiaux), météorites,

et interférences (champs gravitationnels). L'utilisateur doit pouvoir définir un espace 2D de dimensions paramétrables (par défaut 1000x1000 unités), il doit pouvoir placer entre 5 et 20 obstacles statiques (forme circulaire, rayon paramétrable) dans l'environnement. Le système devrait permettre d'ajouter des astres exerçant une attraction gravitationnelle.

Toute trajectoire calculée doit prendre en compte ces éléments pour éviter les collisions ou les déviations non désirées. Le programme doit permettre à l'utilisateur de définir un point de départ et une destination via l'interface. Le système doit calculer une trajectoire optimale, en intégrant des contraintes physiques (gravité, vitesse, énergie). Les calculs doivent être conformes aux lois de la mécanique orbitale et la trajectoire obtenue doit être la plus court possible. Ensuite, Le système affichera graphiquement la trajectoire calculée sur l'interface.

Les satellites doivent former un réseau interconnecté et coopératif, échangeant des données collectées par des antennes. Ces échanges doivent être gérés via des protocoles de communication réseau. Les données collectées doivent être analysées pour optimiser les trajectoires, et utilisées pour estimer la durée des parcours.

Le système doit inclure une gestion de base de données pour stocker les données collectées (positions, historiques des trajectoires, résultats).

L'utilisateur doit pouvoir sauvegarder et charger une configuration (obstacles, points départ/arrivée).

Des météorites mobiles pourraient être ajoutées avec recalcule de trajectoire en temps réel. Le modèle d'IA devra prédire et proposer des trajectoires dynamiques selon les données collectées simultanément au déplacement du satellite concerné.

Besoins non fonctionnels

Le développement doit être exclusivement en Java structuré en architecture orientée objet, les animations doivent être fluide avec un temps de chargement inférieur a 10s, le code doit être documenté (commentaires, Javadoc) pour faciliter la compréhension, la maintenance, et les évolutions futures et l'interface doit être en français avec des messages d'erreur compréhensible. Le code ne doit pas avoir des dépendances a des chemins absolus.

2 Synthèse des besoins

Le programme doit pouvoir modéliser un espace qui comprend divers astres tels que des planètes, lunes, météorites et objets spatiaux ainsi que par des contraintes physiques, permettre la définition d'un espace 2D paramétrable, et ainsi calculer des trajectoires optimales tout en évitant les collisions. L'utilisateur peut définir des obstacles statiques et des astres gravitationnels. Le système doit afficher graphiquement la trajectoire et former un réseau de satellites interconnectés pour échanger des données et optimiser les trajectoires. Une base de données doit gérer les informations collectées, et l'interface doit permettre de sauvegarder et charger des configurations. Le système doit aussi recalculer les trajectoires en temps réel face à des météorites mobiles, en utilisant un modèle d'IA pour proposer des trajectoires dynamiques.

Le développement doit être en Java, orienté objet, avec des animations fluides, un temps de chargement rapide, et une interface en français. Le code doit être bien documenté et ne pas dépendre de chemins absolus.

3 Modèle choisi

Dans ce projet, le modèle choisi consiste en un modèle combinant physique, algorithme d'optimisation de trajectoire ainsi qu'un module optionnel d'intelligence artificielle.

3.1 Modèle Physique

3.1.1 Choix du Référentiel

Le système de simulation repose sur un référentiel cartésien inertiel bidimensionnel $(O, \vec{x}, \vec{y})$. Ce choix présente plusieurs avantages, il permet de modéliser indifféremment des systèmes géocentriques ou héliocentriques et les équations du mouvement s'écrivent naturellement sous forme d'un système différentiel. Dans ce référentiel, tout objet est défini par sa position $\vec{p} = (x, y)$, sa vitesse $\vec{v} = (v_x, v_y)$ et son acceleration $\vec{a} = (a_x, a_y)$.

3.1.2 Système Géocentrique vs Héliocentrique

Caractéristique	**Géocentrique**	**Héliocentrique**
Corps central	Planète	Étoile
Échelle typique	10^4–10^6 km	10^8–10^9 km
Lois applicables	Newton + Kepler	Newton + Kepler

Table 1.1 – *Comparaison des deux systèmes*

3.1.3 Forces Appliquées

Les perturbations externes telles que la pression de radiation solaire ou les effets magnétiques ne sont pas prises en compte. Seule la force gravitationnelle est considérée comme dominante. La loi de gravitation universelle de Newton s'écrit :

$$\vec{F}_{A/B} = -G\frac{m_A m_B}{r^2}\vec{u}_r \tag{1.1}$$

où G est la constante gravitationnelle, r la distance entre les objets et $\vec{u}_r$ le vecteur unitaire entre eux.

Pour un système à N corps, le principe de superposition donne :

$$\vec{F}_{\text{totale}} = \sum_{i \neq j} \vec{F}_{ij} \tag{1.2}$$

3.1.4 Lois de Newton

Première loi (Inertie) Un objet sans force externe conserve une vitesse constante :

$$\vec{p}(t) = \vec{p}_0 + \vec{v}_0 t.$$

Deuxième loi (Dynamique)

$$\vec{F}_{\text{totale}} = m\vec{a}.$$

Troisième loi (Action–Réaction)

$$\vec{F}_{A\to B} = -\vec{F}_{B\to A}.$$

Cette propriété garantit la conservation de la quantité de mouvement.

3.1.5 Lois de Kepler

Les lois de Kepler émergent naturellement de la dynamique newtonienne.

Première loi (Orbites) Les planètes du système solaire décrivent des trajectoires elliptiques, dont le Soleil occupe l'un des foyers.

Deuxième loi (Aires) Les aires balayées par le rayon vecteur d'une planète sont proportionnelles au temps.

Troisième loi (Périodes) Pour toutes les orbites planétaires (satellites) le rapport du carré des périodes de révolution (T en s) au cube du demi-grand-axe de l'orbite (a en m) est constant.

$$T^2 = \frac{4\pi^2}{GM} a^3.$$

$$\frac{T^2}{a^3} = constante$$

Équations du mouvement À partir de l'accélération obtenue, la vitesse et la position des objets spatiaux sont mises à jour numériquement à chaque pas de temps Δt selon les équations suivantes :

$$\vec{v}_{t+\Delta t} = \vec{v}_t + \vec{a} \cdot \Delta t$$

$$\vec{p}_{t+\Delta t} = \vec{p}_t + \vec{v}_{t+\Delta t} \cdot \Delta t$$

Ces équations permettent de simuler la trajectoire des différents objets spatiaux au cours du temps.

3.1.6 Orbite

Dans cette section, nous allons travailler avec les coordonnées polaires (r, θ). Les vecteurs unitaires polaires sont reliés au repère cartésien par les relations suivantes :

$$\vec{e_r} = \cos(\theta)\,\vec{i} + \sin(\theta)\,\vec{j}, \tag{1.3}$$
$$\vec{e_\theta} = -\sin(\theta)\,\vec{i} + \cos(\theta)\,\vec{j}. \tag{1.4}$$

Ainsi, pour un point situé en (r_1, θ_1), les coordonnées cartésiennes s'écrivent :

$$x_1 = r_1 \cos(\theta_1), \tag{1.5}$$
$$y_1 = r_1 \sin(\theta_1). \tag{1.6}$$

Il s'agit simplement de la projection du vecteur position sur les axes du repère cartésien.

D'autre part, le vecteur position s'écrit :

$$\vec{p} = r\,\vec{e_r}. \tag{1.7}$$

En appliquant la règle de dérivation en chaîne, on obtient les dérivées des vecteurs unitaires :

$$\frac{d\vec{e_r}}{dt} = \dot{\theta}\,\vec{e_\theta}, \tag{1.8}$$
$$\frac{d\vec{e_\theta}}{dt} = -\dot{\theta}\,\vec{e_r}. \tag{1.9}$$

La dérivée du vecteur position donne alors la vitesse :

$$\vec{v} = \frac{d\vec{p}}{dt} = \dot{r}\,\vec{e_r} + r\,\dot{\theta}\,\vec{e_\theta}. \tag{1.10}$$

Puis, en dérivant à nouveau :

$$\vec{a_r} = \frac{d\vec{v}}{dt}, \tag{1.11}$$

$$\vec{a} = \left(\ddot{r} - r\,\dot{\theta}^2\right)\,\vec{u_r} + \left(r\,\ddot{\theta} + 2\,\dot{r}\,\dot{\theta}\right)\,\vec{u_\theta} \tag{1.12}$$

on obtient pour rayon constant, après simplification , que la norme de l'accélération est :

$$a = \frac{\|\vec{v}\|^2}{r}. \tag{1.13}$$

Cette relation est caractéristique de l'accélération centripète.

Vitesse nécessaire pour une orbite

Soient A et B deux corps célestes, de masses respectives m_A et m_B. On note r la distance constante entre les deux centres de gravité de A et B. On cherche la vitesse que doit posséder B pour orbiter autour de A.

D'après la deuxième loi de Newton, la force s'écrit :

$$\vec{F}_{B/A} = m_B\, a.$$

D'après la loi universelle de la gravitation :

$$\vec{F}_{B/A} = \frac{G\, m_A m_B}{r^2}.$$

En divisant les deux expressions par m_B, on obtient l'accélération gravitationnelle :

$$a = \frac{F}{m_B} = \frac{G\, m_A}{r^2}.$$

Pour une orbite circulaire stable, cette accélération est égale à l'accélération centripète :

$$\frac{v^2}{r} = \frac{G\, m_A}{r^2}.$$

En multipliant par r et en isolant v, on obtient :

$$v^2 = \frac{G\, m_A}{r} \quad \Longrightarrow \quad v = \sqrt{\frac{G\, m_A}{r}}.$$

Donc, pour obtenir une orbite circulaire :

$$v_{\text{circulaire}} = \sqrt{\frac{G m_A}{r}}. \tag{1.14}$$

La période orbitale T s'obtient à partir de la relation classique entre la période, la vitesse orbitale et le rayon :

$$T = \frac{2\pi r}{v}.$$

En utilisant l'expression de la vitesse orbitale

$$v = \sqrt{\frac{G\, m_A}{r}},$$

on obtient :

$$T = \frac{2\pi r}{\sqrt{\frac{G\, m_A}{r}}} = 2\pi \sqrt{\frac{r^3}{G\, m_A}}.$$

3.1.7 Collisions

On dit qu'il y a collision ou choc entre deux ou plusieurs corps céleste quand ces objets subissent une interaction mutuelle de courte durée et de courte portée. Le choc est localisé dans le temps et l'espace. En règle générale, les forces d'interaction sont négligeables quand les corps céleste sont suffisamment éloignées. On peut donc distinguer un « avant» et un «après» la collision. Les particules se déplacent en ligne droite avec des vitesses uniformes. On notera $\vec{v}_i$ la vitesse d'une particule avant le choc et $\vec{v'}_i$ celle après.

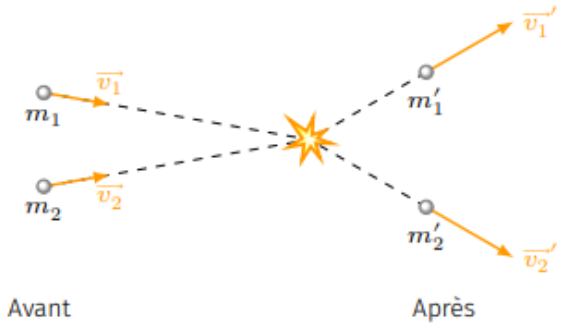


Figure 1.1

On distingue principalement trois type de collisions. **Collision élastique**, on dit qu'il y a collision élastique lorsque le nombre de particules reste constant et que l'énergie interne de chaque particule reste inchangée avant et après le choc. En d'autres termes, les particules ne se déforment pas ni ne changent de nature. **Collision inélastique**, on dit qu'une collision est inélastique lorsqu'une partie de l'énergie cinétique initiale du système s'est transformée en d'autres formes d'énergie. La collision s'accompagne alors d'une variation d'énergie interne et/ou d'une modification du nombre de particules, certaines pouvant être créées par fragmentation ou par équivalence masse-énergie. Enfin, la **collision catastrophique**, on dit qu'une collision est catastrophique lorsque l'énergie mise en jeu est si importante que les particules sont complètement désintégrées ou fragmentées en un grand nombre de particules plus petites. Ce type de collision se produit généralement à très haute énergie et peut entraîner une perte totale de la structure initiale des objets en collision, avec une dissipation maximale de l'énergie cinétique et une redistribution complète de la masse du système.

Dans notre cas, on va traiter les deux derniers types de collision, c'est-à-dire la collision inélastique et catastrophique.

Collision inélastique

Supposons qu'un corps de masse m_1 se déplace à la vitesse v_1, heurte une cible immobile de masse m_2, puis qu'elle se lie a elle. On parle alors de choc mou. Après la collision, l'ensemble se déplace a la vitesse v'.



Figure 1.2

Les lois de conservation s'ecrivent

$$m_1 v_1 = (m_1 + m_2)v'$$

Donc :

$$v' = \frac{m_1}{(m_1 + m_2)} v_1$$

Quand la cible est beaucoup plus massive que le projectile ($m_2 \gg m_1$), on a :

$$\frac{m_2}{m_1 + m_2} \approx \frac{m_2}{m_2} = 1$$

Presque toute l'énergie cinétique du projectile est dissipée ! La vitesse finale de la cible est :

$$v' = \frac{m_1}{m_1 + m_2} v \approx 0$$

La cible ne se déplace pratiquement pas.

En général, on définit le coefficient de restitution est défini comme le rapport entre les vitesses relatives après et avant l'impact.

Choc	**élastique**	**acier-acier**	**bois-bois**	**choc inélastique**
C_R	1	0,95	0,5	0

Table 1.2 – *Coefficients de restitution pour différents types de chocs*

où v_a est la vitesse du premier objet après impact, v_b est la vitesse du second objet après impact, u_a est la vitesse du premier objet avant impact, u_b est la vitesse du second objet avant impact, m_a est la masse du premier objet, m_b est la masse du second objet, C_R est le coefficient de restitution.

On a :

$$v_a = \frac{C_R m_b (u_b - u_a) + m_a u_a + m_b u_b}{m_a + m_b}$$

$$v_b = \frac{C_R m_a (u_a - u_b) + m_a u_a + m_b u_b}{m_a + m_b}$$

On suppose que les collision entre planète est parfaitement inélastique donc C_R est égale à 0. Donc :

$$v_a = \frac{m_a u_a + m_b u_b}{m_a + m_b}$$

$$v_b = \frac{m_a u_a + m_b u_b}{m_a + m_b}$$

3.2 Propulsion

3.2.1 Propulsion spatiale

La propulsion spatiale repose sur le principe fondamental de la troisième loi de Newton. Lorsqu'un satellite expulse une masse de gaz dans une direction, il subit une force opposée qui modifie donc sa vitesse. C'est ainsi qu'un système de propulsion permet d'accélérer ralentir ou bien encore de changer la trajectoire d'un satellite.

On définit ainsi plusieurs grandeurs physiques :

- m_0 la masse initiale en kg (masse de l'engin + masse du carburant)
- m_f la masse finale après consommation du carburant en kg
- $m_c = m_0 - m_f$ la masse du carburant consommé en kg
- v_e la vitesse en m/s
- Isp l'impulsion spécifique en secondes
- $g_0 = 9.80665\ m/s^2$ l'accélération standard de la pesanteur

On a par ailleurs la formule suivante :

$$v_e = Isp \times g_0$$

L'Isp, donc l'impulsion spécifique est un indicateur direct de l'efficacité d'un moteur. Plus l'impulsion spécifique est élevée, moins il faut de carburant pour produire un même changement de vitesse.

3.2.2 Equation de Tsiolkovski

Le changement total de vitesse qu'un satellite peut obtenir grâce à son moteur, appelé ΔV est donné par l'équation fondamentale de Tsiolkovski :

$$\Delta V = v_e \times ln(\frac{m_0}{m_f})$$

Ou encore :

$$\Delta V = Isp \times g_0 \times \frac{m_0}{m_f}$$

Donc, plus le satellite a une masse importante, plus son ΔV disponible augmente et plus l'Isp est grand, plus le ΔV disponible est important pour une même masse.

Ainsi pour trouver la masse consommée lors d'une propulsion, on a ainsi :

$$m_c = m_f \times (e^{\frac{\Delta V}{isp \times g_0}} - 1)$$

3.3 Modèle algorithmique

Le modèle d'optimisation de trajectoire est basé sur un algorithme d'optimisation A^*. Il s'agit d'un algorithme de chemin largement utilisé en navigation autonome, en robotique ou encore en intelligence artificielle par exemple. Il permet d'obtenir un chemin optimal entre un point de départ et une destination tout en évitant les obstacles.

Dans le cadre du projet, l'algorithme A^* présente plusieurs avantages :

- Il fonctionne efficacement dans un espace bidimensionnel
- Il permet de prendre en compte la présence d'obstacles fixes dans l'espace simulé
- Il peut intégrer plusieurs critères tels que la distance, le carburant consommé ainsi que le risque de collision

3.3.1 Fonction de coût

Dans l'algorithme A^* on peut considérer chaque position possible du satellite comme étant représenté par un nœud n associée à une fonction de coût totale $f(n)$ définie par :

$$f(n) = g(n) + h(n)$$

où:

- $g(n)$ représente le coût réel depuis le point de départ jusqu'au nœud n et le point d'arrivé.
- $h(n)$ représente une estimation heuristique du coût restant entre le nœud et le point d'arrivé.

Dans le contexte de ce projet, le coût réel $g(n)$ peut être défini comme tel :

$$g(n) = d(n) + c(n) + p(n)$$

où :

- $d(n)$ est la distance parcourue depuis l'origine
- $c(n)$ est une estimation de l'énergie consommée
- $p(n)$ est une pénalité liée à la proximité des obstacles et aux zones à risques

Par ailleurs pour pouvoir ajouter plus ou moins d'importance à certains critères, une version pondérée de la fonction peut être considérée :

$$f(n) = \alpha \cdot d(n) + \beta \cdot c(n) + \gamma \cdot p(n)$$

où :

— α est un coefficient associé à la distance
— β est un coefficient associé à la consommation énergétique
— γ est un coefficient associé au risque de collision

La fonction heuristique $h(n)$ est la distance euclidienne entre la position actuelle du satellite et la position cible :

$$h(n) = \sqrt{(x_{cible} - x_n)^2 + (y_{cible} - y_n)^2}$$

L'algorithme A^* permet donc de générer une trajectoire optimale pour le satellite, en tenant compte à la fois des lois physiques et des contraintes de l'environnement spatial simulé.

4 Code

ObjetSpatial

La classe **ObjetSpatial** permet de modéliser un objet dans l'espace en 2D. Chaque Objet est défini par sa position,sa masse, son rayon, un identifiant unique ainsi qu'une vitesse selon les axes X et Y. Elle encapsule donc les propriétés physiques et le mouvement d'un objet spatial.

```
1
2  public void updatePosition(double dt) {
3        Point2D position = this.getPosition();
4
5        double nouveauX = position.getX() + this.getVelocityX() * dt;
6        double nouveauY = position.getY() + this.getVelocityY() * dt;
7
8        this.setPosition(new Point2D.Double(nouveauX, nouveauY));
9     }
```

Listing 1.1 – *Classe ObjetSpatial en Java*

Il est nécessaire par ailleurs de mettre à jour la position, de ce fait on met en place la méthode **updatePosition** qui déplace l'objet en fonction du temps. On utilise ainsi les formules de cinématique de base :

$$x_1 = x_0 + v_x.\Delta t$$
$$y_1 = y_0 + v_y.\Delta t$$

Lune

Une lune est un **ObjetSpatial** qui possède les propriétés spécifiques tels qu'un nom, une gravité propre, une planète parent, une période de révolution et un rayon orbital. On applique ici le principe de l'héritage en programme orientée objet.

```
Point2D posParent = planeteParente.getPosition();
double dx = position.getX() - posParent.getX();
double dy = position.getY() - posParent.getY();
this.rayonOrbital = Math.sqrt(dx * dx + dy * dy);
```

Listing 1.2 – *Classe Lune en Java*

On définit aussi le rayon orbital de la planète comme la distance euclidienne ci contre :

$$\sqrt{(x_{lune} - x_{planete})^2 + (y_{lune} - y_{planete})^2}$$

Planète

La classe **Planete** permet de modéliser une planète dans la simulation spatiale ; elle hérite directement de la classe **ObjetSpatial**, et il lui est ajouté les attributs nom ainsi que la gravité. Pour que son identifiant soit bien unique, on hache le nom avec la méthode *hashcode()* méthode définie dans la classe **Object**

Astéroïde

La classe **Asteroides** permet de modéliser un satellite dans la simulation spatiale ; elle hérité également directement de la classe **ObjetSpatial** et elle possède en plus les attributs nom ainsi que taille.

Satellites

La classe **Satellites** étend la classe **ObjetSpatial** afin de simuler un satellite artificiel capable de se déplacer de manière autonome dans l'espace.

La classe **Satellites** hérite de la classe **ObjetSpatial** et elle ajoute des comportements spécifiques tels que :

- La gestion du carburant
- La gestion d'une trajectoire
- La détection d'astres
- La communication via une antenne
- les manœuvres orbitales(propulsion, freinage, mise en orbite)
- Le déplacement progressif

Elle contient donc la logique du mouvement et de navigation du satellite.

```
public class Satellites extends ObjetSpatial{

    private Antenne antenne;
    private Trajectoire trajectoire;
    private double carburant;
    private Point2D destination;
    private double porteeCamera;
    private double dureeEstimee;
```

```
    private double distanceparcourue;
    private List<ObjetSpatial> astresDetectes;
    private Point2D positionFinale;
    private double isp;
    private double go;
    private int indexWaypointActuel = 0;
    private Set<Integer> objetsDejaEnvoyes = new HashSet<>();
    private ObjetSpatial astreOrbite;
    private double rayonOrbital;
    private boolean enOrbite = false;
```

Listing 1.3 – *Classe Lune en Java*

Les attributs ci-contres représentent l'état du satellite à tout instant. Les satellites possèdent une antenne permettant la communication et une trajectoire représentant le chemin que le satellite doit suivre.

Les attributs peuvent être rassemblés en 4 catégories :

- Composantes externes(BaseDeDonnees,Antenne,Trajectoire)
- Attributs physiques (carburant, destination, porteeCamera, distanceparcourue et dureeEstimee, Isp, g0)
- Détection d'objets spatiaux(astresDetectes, objetsDejaEnvoyes)
- Dynamique de déplacement (indexWaypointActuel qui indique le point de passage actuellement visé, astreOrbite l'astre autour duquel le satellite peut être placé en orbite, rayonOrbital et enOrbite booléen permettant de savoir si le satellite est actuellement en orbite)

Pour un souci de rigueur, le déplacement du satellite est modélisé par la propulsion. Les deux variables que sont l'impulsion spécifique Isp et la gravité de référence $g0$ sont deux variables qui sont utilisés dans l'équation de Tsiolkovski qui permettent de déterminer un ΔV, c'est-à-dire la variation maximale de vitesse réalisable avec le carburant restant. Cette équation constitue ainsi une base fondamentale pour estimer la capacité de manœuvre du satellite. On a ainsi les méthodes suivantes :

ΔV et carburant consommé

```
public double deltaVDisponible() {
        return isp * go * Math.log((getMasse()+ getCarburant()) / getMasse());
    }

    public double carburantNecessaire(double deltaV) {
        return (getMasse() +getCarburant())* (1 - Math.exp(-deltaV / (isp*go)));

    }
```

Listing 1.4 – *Classe Lune en Java*

Il est ainsi implémenté d'après les équations de Tsiolkovski :

$$\Delta V = Isp \times g_0 \times \frac{m_0}{m_f}$$

$$m_c = m_f \times (e^{\frac{\Delta V}{isp \times g_0}} - 1)$$

pour déterminer le ΔV disponible ainsi que le carburant consommé lors de l'application d'une manœuvre.

Orbite satellite

Pour pouvoir placer en orbite un Satellite, partant de l'hypothèse que ce dernier ne consomme pas de carburant car il a une vitesse orbitale précise qui le fait orbiter autour de l'astre la méthode *placerEnOrbite* positionne le satellite sur une orbite circulaire autoure d'un astre à une distance qui est le rayon orbital et calcule la vitesse orbitale qui correspond.

```
double angle,x,y;
      double[] vitesseOrbitale;
      this.astreOrbite = astre;
      this.rayonOrbital = rayonOrbital;
      this.enOrbite = true;
      angle = Math.random() * 2 * Math.PI;
      x = astre.getPosition().getX() + rayonOrbital * Math.cos(angle);
      y = astre.getPosition().getY() + rayonOrbital * Math.sin(angle);

      this.setPosition(new Point2D.Double(x, y));
      vitesseOrbitale = ForceGravitationnelle.calculerVitesseOrbitale(astre,
          this.getPosition(), rayonOrbital);
      this.setVelocite(vitesseOrbitale[0], vitesseOrbitale[1]);
```

Listing 1.5 – *Classe Lune en Java*

On place ainsi le satellite en les coordonnées déterminées par un angle aléatoire θ :

$$x = x_{astre} + rcos(\theta)$$

$$y = y_{astre} + rsin(\theta)$$

et on détermine la vitesse orbitale en utilisant la méthode *calculerVitesseOrbitale* de la classe **ForceGravitationnelle**. Elle détermine la vitesse orbitale par :

$$v_{circulaire} = \sqrt{\frac{Gm_A}{r}}$$

avec la direction tangente au rayon.

Déplacement du satellite

Pour pouvoir gérer le déplacement du satellite selon la trajectoire donnée, plusieurs problèmes ont d'abord été rencontrés : le satellite suivait simplement la ligne de Points de la trajectoire sans réelle physique derrière, sans propulsion ou bien manœuvre. Pour pouvoir remédier à ce problème, la méthode *deplacerSatelliteProgressif* fait avancer le satellite le long d'une trajectoire composée de "waypoints", des différents points de la trajectoire. Elle gère ainsi la navigation progressive en détectant l'arrivée au prochain waypoint, le calcul de forces que sont manoeuvres et propulsion et une application de la consommation du carburant. C'est une méthode qui renvoie un booléen exprimant si on arrive à la destination finale ou non.

```
1  public boolean deplacerSatelliteProgressif(List<Point2D> trajectoire, double dt)
       {
2        double manoeuvreX,manoeuvreY,propulsionX,propulsionY;
3         if (!validerConditionsDeplacement(trajectoire)) {
4             return true;
5         }
6         Point2D waypointCible = trajectoire.get(indexWaypointActuel);
7         Point2D positionActuelle = getPosition();
8
9         if (estArriveAuWaypoint(waypointCible, positionActuelle)) {
10            return gererArriveeWaypoint(dt);
11        }
12        double[] forces = calculerForcesDeDeplacement(waypointCible,
              positionActuelle);
13        manoeuvreX = forces[0];
14        manoeuvreY = forces[1];
15        propulsionX = forces[2];
16        propulsionY = forces[3];
17
18        appliquerForcesAvecConsommation(manoeuvreX, manoeuvreY, propulsionX,
              propulsionY, dt);
19
20        mettreAJourPosition(dt);
21
22        return false;
23    }
```

Listing 1.6 – *Classe Lune en Java*

Il est ainsi validé les condition de déplacement via *validerConditionsDeplacement*. Par la suite on récupère le prochain waypoint de la trajectoire, si le satellite est déjà arrivé au waypoint, son comportement est géré par la méthode *gererArriveAuWaypoint* sinon on calcule les forces de déplacements et on applique ces forces avec la consommation de carburant qui va avec. Finalement, on met à jour la position.

Gérer arrivée au waypoint et freinage

```java
private boolean gererArriveeWaypoint(double dt) {
        if (indexWaypointActuel == getTrajectoire().getPoints().size() - 1) {
            return appliquerFreinageArrt(dt);
        } else {
            indexWaypointActuel++;
            return false;
        }
    }

private boolean appliquerFreinageArrt(double dt) {
        double vitesseActuelle, directionFreinX,directionFreinY;

        vitesseActuelle = norme(new double[]{getVelociteX(),getVelociteY()} );

        if (vitesseActuelle < 0.1) {
            arreterSatellite();
            indexWaypointActuel++;
            return true;
        }
        directionFreinX = -getVelociteX() / vitesseActuelle;
        directionFreinY = -getVelociteY()/ vitesseActuelle;

        // Intensit du freinage
        double accelerationFrein = Math.min(vitesseActuelle / dt, 30.0);
        double accelerationX = directionFreinX * accelerationFrein;
        double accelerationY = directionFreinY * accelerationFrein;

        double deltaV = accelerationFrein*dt;
        double consommation = carburantNecessaire(deltaV);

        if (carburant >= consommation) {
            setVelociteX(getVelociteX() + accelerationX * dt);
            setVelociteY(getVelociteY() + accelerationY * dt);
            carburant -= consommation;
        } else if (carburant > 0) {
            // Freinage partiel avec le carburant restant
            double ratio = carburant / consommation;
            setVelociteX(getVelociteX() + accelerationX * dt * ratio);
            setVelociteY(getVelociteY() + accelerationY * dt * ratio);
            carburant = 0;
        }

        Point2D positionActuelle = getPosition();
        setPosition(new Point2D.Double(positionActuelle.getX() + getVelociteX() *
             dt, positionActuelle.getY() + getVelociteY() * dt));

        return false;
    }
```

Listing 1.7 – *Classe Lune en Java*

Ces deux méthodes permettent de gérer le passage de waypoint en waypoint jusqu'au dernier. On incrémente simplement le nombre de waypoint tant qu'on n'est pas arrivé au dernier et lorsqu'on arrive au dernier, on freine avec la méthode *appliquerFreinageArrêt*. La méthode *appliquerFreinageArrêt* dans un premier temps normalise les composantes opposées de vitesse pour obtenir un vecteur direction unitaire, et ainsi permettre le calcul de l'accélération à appliquer pour freiner. L'accélération s'exprime de la sorte : $a = \frac{v_{actuelle}}{dt}$ Ainsi à chaque pas de temps dt on détermine la vitesse actuelle pour s'arrêter de manière rapide et progressive et non pas d'un coup ce qui serait irréaliste. Par ailleurs on limite arbitraiement l'accélération à 30 unités pour éviter que le freinage soit irréaliste. En effet, un satellite ne peut pas générer une force infinie instantanément. On converit par la suite la norme de l'accélération en composantes X et Y selon la direction du freinage pour que chaque composante soit proportionnelle au vecteur unitaire directionnel. Puis on fait une approximation du changement de vitesse ΔV sur un petit pas de temps. Etant donné que le pas de temps est petit, il n'est pas nécessaire d'intégrer tel que :

$$\Delta V_{total} = \int_0^T a(t)dt$$

On utilise ainsi par la suite *carburantNecessaire* qui calcule la masse nécessaire pour produire ce ΔV.

Puis pour la consommation de carburant pour appliquer le freinage complet, deux scénarios sont posés :

- Soit on applique l'accélération complète sur dt et on réduit le carburant de la quantité nécessaire quand on a assez de carburant pour réaliser un freinage complet
- Soit on n'a pas assez de carburant pour un freinage complet, on applique un freinage partiel proportionnel à ce qu'il reste en utilisant un ratio qui permet au satellite de freîner autant qu'il le peut avant le carburant restant.

La position du satellite est toujours mise à jour.

4.0.1 Propulsion, Manoeuvre, Forces de déplacements

```
1  private double[] calculerForcesDeDeplacement(Point2D waypointCible, Point2D
       positionActuelle) {
2          double dx,dy,distance,vitesseX,vitesseY,vitesseActuelle;
3          double[] forces, manoeuvres, propulsion;
4
5          forces = new double[4]; // [manoeuvreX, manoeuvreY, propulsionX,
               propulsionY]
6
7          // Calcul des vecteurs de base
8          dx = waypointCible.getX() - positionActuelle.getX();
9          dy = waypointCible.getY() - positionActuelle.getY();
10         distance = norme(new double[]{dx,dy});
```

```
11
12          vitesseX = getVelociteX();
13          vitesseY = getVelociteY();
14          vitesseActuelle = norme(new double[]{vitesseX,vitesseY});
15          // Calcul des manoeuvres de correction de trajectoire
16
17          manoeuvres = calculerManoeuvresCorrection(dx, dy, distance, vitesseX,
                vitesseY, vitesseActuelle);
18          forces[0] = manoeuvres[0];
19          forces[1] = manoeuvres[1];//manoeuvre et propulsion  ajouter
20
21          // Calcul de la propulsion principale
22          propulsion = calculerPropulsionPrincipale(dx, dy, distance,
                vitesseActuelle, (indexWaypointActuel == getTrajectoire().getPoints()
                .size() - 1));
23          forces[2] = propulsion[0];
24          forces[3] = propulsion[1];//force  quoi a sert
25
26          return forces;
27      }
28
29      private double[] calculerManoeuvresCorrection(double dx, double dy, double
            distance, double vitesseX, double vitesseY, double vitesseActuelle) {
30          double manoeuvreX,manoeuvreY,directionSouhaiteeX,directionSouhaiteeY,
                directionActuelleX,directionActuelleY,agressiviteManoeuvre;
31
32          manoeuvreX = 0;
33          manoeuvreY = 0;
34
35          if (vitesseActuelle > 0.1) {
36              directionSouhaiteeX = dx / distance;
37              directionSouhaiteeY = dy / distance;
38              directionActuelleX = vitesseX / vitesseActuelle;
39              directionActuelleY = vitesseY / vitesseActuelle;
40
41              double produitScalaire = directionActuelleX * directionSouhaiteeX +
                    directionActuelleY * directionSouhaiteeY;
42              double angle = Math.acos(Math.max(-1, Math.min(1, produitScalaire)));
                    // voir
43
44
45              if (angle>Math.PI/4){
46                  agressiviteManoeuvre=2.5;
47              }
48              else {agressiviteManoeuvre=1;}
49
50              if (angle > Math.PI / 180) {
```

```
51                double intensiteManoeuvre = Math.min(angle * 3.0 *
                      agressiviteManoeuvre, 2.0);
52
53                double normePerpX = -directionActuelleY;
54                double normePerpY = directionActuelleX;
55
56                double determinant = directionActuelleX * directionSouhaiteeY -
                      directionActuelleY * directionSouhaiteeX;
57                if (determinant < 0) {
58                    normePerpX = -normePerpX;
59                    normePerpY = -normePerpY;
60                }
61
62                manoeuvreX = normePerpX * intensiteManoeuvre * 25.0 *
                      agressiviteManoeuvre;
63                manoeuvreY = normePerpY * intensiteManoeuvre * 25.0 *
                      agressiviteManoeuvre;
64            }
65
66            if (angle > Math.PI / 180 && vitesseActuelle > 50) {
67                appliquerRalentissementVirage();
68            }
69        }
70
71        return new double[]{manoeuvreX, manoeuvreY};
72    }
73
74    private double[] calculerPropulsionPrincipale(double dx, double dy, double
          distance, double vitesseActuelle, boolean estDernierWaypoint) {
75        double propulsionX,propulsionY,intensitePropulsion;
76
77        propulsionX = 0;
78        propulsionY = 0;
79
80        if (estDernierWaypoint && distance < 300) {
81            intensitePropulsion = Math.min(distance / 300.0, 1.0) * 0.3;
82        } else {
83            double facteurVirage = calculerFacteurVirage(dx, dy, distance,
                  vitesseActuelle);
84            intensitePropulsion = Math.min(distance / 100.0, 1.0) * 0.5 *
                  facteurVirage;
85        }
86
87        if (distance > 30 || vitesseActuelle < 25) {
88            double directionX = dx / distance;
89            double directionY = dy / distance;
90
91            propulsionX = directionX * intensitePropulsion * 12.0;
```

```
92              propulsionY = directionY * intensitePropulsion * 12.0;
93          }
94
95          return new double[]{propulsionX, propulsionY};
96      }
```

Listing 1.8 – *Classe Lune en Java*

Le déplacement du satellite est donné en combinant des manoeuvres correctives, donc des ajustements directionnels ainsi qu'une propulsion principale. Dans un premier temps, on détermine la distance euclidienne entre la position actuelle du satellite et la position du prochain waypoint ains ique sa vitesse. Ces variables sont ensuites utilisées dans la classe gérant les maneouvre qui visent à ajuster la direction du satellite pour qqu'elle coïncide avec la trajectoire optimale vers le waypoint. Pour pouvoir réaliser une manoeuvre, il est nécessaire de calculer l'angle entre la direction acutelle et la direction souhaitée; on prend ainsi le produit scalaire pour déterminer l'angle θ nécessaire auquel on applique $arccos(prodscal)$ et on pose le minimum entre -1 et 1 au cas où des erreurs de flottants aient lieux. On rappelle que le produit scalaire s'exprime de la sorte :

$$\vec{u} \cdot \vec{v} = \|\vec{u}\| \, \|\vec{v}\| \, \cos\theta$$

Pour savoir dans quel sens on utilise la correction on utilise un déterminant qui nous donne le côté sur lequel appliquer la manœuvre. Si le déterminant est positif on tourne vers le sens horaire, si négatif vers le sens anti-horaire. On détermine par ailleurs si on doit avoir une correction agressive lorsque l'angle calculé est plus grand que $\frac{\pi}{4}$. L'intensité de la manœuvre est proportionnelle à l'angle, modulée par un facteur d'agressivité qui augmente donc si l'angle est supérieur à 45. Si le satellite se déplace rapidement, il doit tourner fortement et ainsi on a besoin d'utiliser la méthode *appliquerRalentissementVirage* qui va réduire temporairement la vitesse afin de limiter le risque de dépassement de trajectoire.

La propulsion permet d'avancer réellement vers le prochain waypoint. La méthode permet de calculer l'intensité normalisée de la propulsion. Elle renvoie un vecteur des deux composantes de propulsions en X et en Y qui représentent l'accélération à appliquer pour avancer vers la cible. La propulsion pousse le satellite en avant. On cherche donc à calculer une intensité normalisée de propulsion, on regarde deux cas, si on arrive au dernier waypoint, on propulse en douceur. Sinon dans le cas général, l'intensité de base est proportionnelle à la distance qui est normalisée. On multiplie arbitrairement par des valeurs tels que 0.3 ou encore 0.5 et encore un facteur de virage qui provient de la méthode $calculerFacteurVirage$ qui sert à diminuer la propulsion en cas d'angle important entre vitesse actuelle et direction souhaitée, afin d'éviter d'accélérer tandis que l'on doit pivoter. L'intensité de propulsion dépend essentiellement de la distance jusqu'au prochain waypoint.

Trajectoire

determinerTrajectoire

```java
public void determinerTrajectoire(Satellites satellites) {
    if (pointDepart == null || pointArrivee == null) {
        estValide = false;
        setPoints(new ArrayList<>());
    }

    if (distanceEuclidienne(pointDepart, pointArrivee)>8000){
        noeud.PAS_DEPLACEMENT=320;
        noeud.TOLERANCE_ARRIVEE=350;
    }
    else if (distanceEuclidienne(pointDepart, pointArrivee) > 5000) {
        noeud.PAS_DEPLACEMENT = 220;
        noeud.TOLERANCE_ARRIVEE = 240;
    }

    else if (distanceEuclidienne(pointDepart, pointArrivee) > 2000) {
        noeud.PAS_DEPLACEMENT = 100;
        noeud.TOLERANCE_ARRIVEE = 110;
    }
    else if (distanceEuclidienne(pointDepart, pointArrivee) > 1000) {
        noeud.PAS_DEPLACEMENT = 50;
        noeud.TOLERANCE_ARRIVEE = 60;
    }
    else if (distanceEuclidienne(pointDepart, pointArrivee)>500){
        noeud.PAS_DEPLACEMENT = 28;
        noeud.TOLERANCE_ARRIVEE = 36;
    }
    else if (distanceEuclidienne(pointDepart,pointArrivee)>200){
        noeud.PAS_DEPLACEMENT=18;
        noeud.TOLERANCE_ARRIVEE=20;
    }
    else{
        noeud.PAS_DEPLACEMENT=10;
        noeud.TOLERANCE_ARRIVEE=12;
    }

    PriorityQueue<Noeud> listeOuverte = new PriorityQueue<>();
    Set<Noeud> listeFermee = new HashSet<>();
    Map<Point2D, Double> meilleursScoresG = new HashMap<>();

    Noeud noeudDepart = new Noeud(pointDepart, null);
    noeudDepart.g = 0;
    noeudDepart.h = calculerH(pointDepart);
    noeudDepart.f = noeudDepart.h;

    listeOuverte.add(noeudDepart);
    meilleursScoresG.put(pointDepart, 0.0);

```

```
49      Noeud noeudFinal = null;
50      int iterations = 0;
51      int maxIterations = (int) Math.max(10000, distanceEuclidienne(pointDepart,
            pointArrivee) * 10);
52
53      while (!listeOuverte.isEmpty() && iterations < maxIterations) {
54          iterations++;
55          Noeud noeudCourant = listeOuverte.poll();
56          if (distanceEuclidienne(noeudCourant.position, pointArrivee) < noeud.
                TOLERANCE_ARRIVEE) {
57              noeudFinal = noeudCourant;
58              break;
59          }
60
61          listeFermee.add(noeudCourant);
62
63          for (Noeud voisin : obtenirVoisins(noeudCourant, satellites)) {
64              if (listeFermee.contains(voisin)) {
65                  continue;
66              }
67
68              double G1 = noeudCourant.g + noeud.alpha * distanceEuclidienne(
                    noeudCourant.position, voisin.position) + noeud.beta *
                    calculerCoutEnergetique(satellites,noeudCourant.position, voisin.
                    position) + noeud.gamma * calculerPenalite(voisin.position,
                    satellites);
69              Double G0 = meilleursScoresG.get(voisin.position);
70
71              if (G0 == null || G1 < G0) {
72                  voisin.parent = noeudCourant;
73                  voisin.g = G1;
74                  voisin.h = calculerH(voisin.position);
75                  voisin.f = voisin.g + voisin.h;
76                  meilleursScoresG.put(voisin.position, G1);
77                  if (!listeOuverte.contains(voisin)) {
78                      listeOuverte.add(voisin);
79                  }
80              }
81          }
82      }
83      if (noeudFinal != null) {
84          List<Point2D> chemin = reconstruireChemin(noeudFinal);
85          this.points = chemin;
86          calculerStatistiques(chemin, satellites);
87          this.estValide = true;
88          setPoints(chemin);
89      } else {
90          this.estValide = false;
```

```
91          setPoints(new ArrayList<>());
92      }
93  }
```

Listing 1.9 – *Classe ForceGravitationnelle en Java*

La méthode *determinerTrajectoire* de la classe **Trajectoire** permet la recherche de trajectoire en utilisant une méthode A^* sur un graphe implicite défini par un maillage circulaire autour de chaque noeud (16 directions). On utilise trois structures de données importantes:

- Une **PriorityQueue**
- Un **Set**
- Une **Map**

Elles permettent de choisi efficacement quel nœud explorer, d'éviter les répétitions et d'optimiser le chemin trouvé.

La **PriorityQueue** contient les noeuds à explorer ; c'est une structure de données qui stocke des éléments en fonction d'une priorité. Dans notre cas, on cherche le coût total minimal estimé de la trajectoire, c'est pour cela qu'on récupère le noeud avec le plus petit f automatiquemenent par la méthode *poll*.

Pour le **Set** utilisé qui est une collection qui ne peut contenir de doublons, on stocke tous les noeuds déjà explorés. Cela permet d'éviter de repasser plusieurs fois par le même point, de réduire le temps de calcul et d'empêcher les boucles infinies.

Pour la **Map** qui associe une clé à une valeur, on associe une position au meilleur coût g trouvé jusqu'à maintenant pour cette position. Cela permet de garder uniquement les meilleurs chemins et ainsi d'appliquer le principe fondamental de l'algorithme A^*.

Dans ce code dans un premier temps, on règle de manière adaptative le pas de déplacement et la tolérance d'arrivée en fonction des distances entre le point de départ et le point d'arrivé. Pour les longues distances on augmente le pas, on a ainsi un maillage plus grossier et pour de courtes distances on utilise un pas plus fin pour une meilleure précision.

Pour la création du nœud de départ, on initialise le coût depuis le départ via la distance euclidienne qui est l'heuristique. Le nœud de départ est inséré dans la **PriorityQueue** et le meilleur score pour f est initialisé pour la position de départ.

On passe par la suite dans la boucle principale dans laquelle on pose un nombre d'itérations maximales afin d'éviter une boucle infinie, elle est proportionnelle à la distance. On renvoie par la suite le nœud de plus petite valeur f. On explore ainsi en priorité les chemins ayant le meilleur potentiel. On vérifie si la distance du nœud courant au point d'arrivé est inférieur à la tolérance d'arrivée, la tolérance qui dit qu'on est suffisamment proche du point d'arrivé. On ajoute ensuite un nœud courant au **set** et on marque ce nœud comme traité pour ne pas l'explorer à nouveau. On explore ensuite les voisins en générant une collections de nœuds voisins via la méthode *obtenirVoisins* et on ignore ceux déjà dans le **Set**. Et pour chaque voisin possible on calcule le nouveau coût candidat comme la somme pondérée par des facteurs arbitraires choisis pour donner plus de poids soit à

la distance, au coût énergétique ainsi qu'à un risque défini par la force gravitationnelle de l'astre. Plus elle est importante, plus la pénalité est importante. Puis on compare le nouveau coût candidat à l'ancien et si il est meilleur on met à jour g, h, f. A la fin, lorsqu'on détermine le noeud final, on remonte la chaîne de parent depuis le noeud final jusqu'au départ pour reconstruire la liste de positions dans un ordre correct. Et par la suite on calcule la longueur, le coût énergétique et la durée estimée. Le coût énergétique dépend du carburant consommé.

ForceGravitationnelle

La classe **ForceGravitationnelle** est la classe qui traite les calculs de force gravitationnelle, elle fournit des méthodes de classe pour calculer les forces, vitesses orbitales necessaire et périodes.

```
1    public static double[] calculerVitesseOrbitale(ObjetSpatial centre, Point2D
         positionOrbite, double rayonOrbital) {
2        Point2D posCentre = centre.getPosition();
3
4        double dx = positionOrbite.getX() - posCentre.getX();
5        double dy = positionOrbite.getY() - posCentre.getY();
6        double distance = Math.sqrt(dx * dx + dy * dy);
7
8        double r = (rayonOrbital > 0) ? rayonOrbital : distance;
9        double vitesseOrbitale = Math.sqrt(G * centre.getMasse() / r);
10       double ux = dx / distance;
11       double uy = dy / distance;
12       double vx = -vitesseOrbitale * uy;
13       double vy = vitesseOrbitale * ux;
14       return new double[]{vx, vy};
15   }
16
17   public static double calculerPeriodeOrbitale(ObjetSpatial centre, double
         rayonOrbital) {
18       return 2 * Math.PI * Math.sqrt(Math.pow(rayonOrbital, 3) / (G * centre.
             getMasse()));
19   }
20   public static double[] calculerForce(ObjetSpatial a, ObjetSpatial b) {
21       Point2D posA = a.getPosition();
22       Point2D posB = b.getPosition();
23       double dx = posB.getX() - posA.getX();
24       double dy = posB.getY() - posA.getY();
25       double distanceSquared = dx * dx + dy * dy;
26
27       // viter la division par zro
28       if (distanceSquared < 1.0) {
29           return new double[]{0, 0};
30       }
31       double distance = Math.sqrt(distanceSquared);
```

```
32          double forceMagnitude = G * a.getMasse() * b.getMasse() / distanceSquared
                ;
33          double forceX = forceMagnitude * (dx / distance);
34          double forceY = forceMagnitude * (dy / distance);
35          return new double[]{forceX, forceY};
36      }
37
38      public static void appliquerForce(ObjetSpatial objet, double[] force, double
             dt) {
39          double masse = objet.getMasse();
40          double ax = force[0] / masse;
41          double ay = force[1] / masse;
42          double nouveauVx = objet.getVelociteX() + ax * dt;
43          double nouveauVy = objet.getVelociteY() + ay * dt;
44          objet.setVelocite(nouveauVx, nouveauVy);
45      }
46  }
```

Listing 1.10 – *Classe ForceGravitationnelle en Java*

4.1 Collision

Collision : Détection, Calculs Physiques et création de débris

La classe *Collision* constitue l'outil central chargé de fournir les bases physiques et mathématiques nécessaires à la gestion des collisions. Elle assure la détection des impacts entre objets spatiaux, et réalise l'ensemble des calculs physiques associés, tels que la vitesse relative, le centre d'inertie, la normale de collision, les angles d'impact et les ratios de masse. Elle gère aussi la génération des débris qu'il s'agisse de collisions légères, inélastiques ou catastrophiques. Les modèles de fragmentation s'appuient sur une répartition de masse aléatoire, des vitesses générées dans des plages définies et une orientation déterminée par la normale de collision tout en appliquant les lois de conservation de la quantité de mouvement c'est à dire lorsqu'il y a conservation, l'addition vectorielle des quantités de mouvement qui sont le produits de la masse du corps et son vecteur vitesse de chaque corps, cette quantité conserve la même valeur avant et après la collision. Plusieurs seuils permettent d'adapter le comportement du système : une vitesse de choc mou fixée à 5 unités, une vitesse de choc catastrophique définie à 40 unités et un ratio de masse négligeable de 50, les choix des seuils ont été réalise lors des tests du code, ils sont arbitraire et peuvent être améliore.

ResolveurCollisionsPhysiques

La classe *ResolveurCollisionsPhysiques* prend en charge les collisions entre objets tels que les astéroïdes ou les planètes.Elle gère plusieurs types de collisions, allant du choc, aux collisions inélastiques jusqu'aux collisions catastrophiques provoquant une fragmentation complète. Les collisions sont classées selon la vitesse relative et le ratio de masse. Elle gère les collisions douces en fusionnant les masses et en calculant la vitesse résultante selon

la conservation de la quantité de mouvement. Les collisions inélastiques produisent une perte d'énergie et la génération contrôlée de débris représentant environ 10% de la masse totale, répartis en fragments. Les collisions catastrophiques, quant à elles, entraînent la fragmentation complète des objets et la création d'un nuage important de débris à haute énergie. La classe calcule systématiquement les vitesses post-collision en utilisant les formules démontres dans la section modèle physique et applique également une séparation afin d'éviter toute recollision immédiate et génère des trajectoires de débris corrélé avec la direction du choc.

GestionnaireSatellites

Le *GestionnaireSatellites* s'occupe exclusivement des collisions impliquant des satellites artificiels. Le comportement dépend du ratio de masse : lorsque les masses sont comparables (ratio inférieur à 5), la collision entraîne une destruction mutuelle avec fragmentation. Sinon lorsqu'on a une collision avec un objet massif (ratio > 5) le satellite est detruit. Ce gestionnaire déclenche automatiquement la déconnexion du satellite détruit du réseau, retire l'objet de la simulation et génère des débris.

GestionnaireCollisions

Le *GestionnaireCollisions* coordonne l'ensemble du processus. Il détecte toutes les collisions grâce aux méthodes fournis par *Collision*, puis les redirige vers la classe approprié : gestionnaire de satellites ou ResolveurCollisionsPhysiques. Une fois la résolution effectuée, il met à jour la liste des objets spatiaux, ajoute les débris, supprime les objets détruits et assure la cohérence globale.

4.2 ForceGravitationnelle

La classe `ForceGravitationnelle` met en œuvre les lois cite dans le modèle physique choisie . Elle effectue tous les calculs pour les interactions gravitationnelles. La valeur de la constante gravitationnelle, représentée par $G = 0.01$, est modifiée selon les dimensions du monde en simulation. Cette valeur assure la stabilité numérique des opérations mathématiques. On détermine l'accélération en divisant la force globale par la masse de l'objet. Ensuite, la vitesse est mise à jour en fonction de cette accélération et de l'intervalle de temps, puis la nouvelle vitesse est utilisée pour mettre à jour le parcours. La vitesse orbitale circulaire est calculée selon la relation $v = \sqrt{\frac{GM}{r}}$ démontre dans la section modèle physique. La force gravitationnelle entre deux objets est définie par $F = G\frac{m_1 m_2}{r^2}$. La force totale appliquée à un objet est obtenue par la somme des contributions de tous les autres objets. La période orbitale est déterminée à l'aide de la troisième loi de Kepler, exprimée par $T = 2\pi\sqrt{\frac{r^3}{GM}}$. Cette liaison facilite la validation des trajectoires calculées et garantit une concordance temporelle entre différentes orbites.

4.3 RéseauSatellites - Système de Communication

La classe **ReseauSatellites** met en place un serveur centralisé assurant la coordination et l'échange de données entre les satellites en temps réel. Elle garantie la diffusion

d'informations.

Les attributs de cette classe incluent un port d'écoute, un indicateur **marche** déclaré **volatile** afin d'assurer une visibilité constante entre les différents threads. L'ensemble des données reçues est stocké dans une structure de type **ConcurrentHashMap**, pour une gestion sécurisée dans un environnement multi-thread.

L'architecture du serveur repose sur un modèle multi-thread dans lequel chaque satellite connecté dispose de son propre thread de communication. Le serveur assure une écoute continue sur son socket principal grâce une boucle tout en traitant les interruptions. Avec l'utilisation du mécanisme *try-with-resources*, les ressources réseau, comme les sockets, sont constamment fermées, garantissant ainsi une libération sure.

Plusieurs commandes structurent le protocole. La commande `DATA <idSatellite> <données>` est utilisée pour transmettre les objets détectés. La commande `GET_OBSTACLES` renvoie l'ensemble des objets actuellement enregistrés, la commande `CLEAR_OBSTACLES` permet de réinitialiser totalement le registre.

Les données échangées sont sérialisées sous un format texte compact, de type `id,x,y,vx,vy,masse,rayon;...`. Les champs sont séparés par des virgules et les objets successifs par des points-virgules. Tous les échanges utilisent un encodage UTF-8 accompagné d'un buffering afin d'améliorer les performances.

Le stockage centralisé s'appuie sur une map thread-safe associant chaque identifiant d'objet à sa représentation spatiale. En cas de doublons, les entrées déjà présentes sont automatiquement remplacées. L'accès concurrent est entièrement pris en charge par `ConcurrentHashMap`.

Le traitement d'une commande `DATA` commence par la réception d'une chaîne sérialisée, suivie d'une désérialisation des objets. Dans un premier temps es doublons sont repère puis s'en suit une mis à jour via la commande `put()`. Le serveur renvoie ensuite un résumé indiquant le nombre d'objets nouveaux ou modifiés.

Lors d'une requête `GET_OBSTACLES`, le serveur vérifie d'abord la présence d'objets dans le registre, sérialise l'ensemble des entrées puis envoie une réponse formatée au client.

Chaque satellite dispose d'une session indépendante, ce qui assure un isolement complet des communications. Les exceptions d'E/S sont journalisées sans provoquer l'arrêt du serveur. La fermeture des ressources est assurée dans les blocs `finally`, et le parsing des données reste tolérant aux erreurs de format.

La sérialisation transforme chaque objet spatial en une chaîne comprenant son identifiant, sa position, sa vitesse, sa masse et son rayon, toutes les valeurs étant séparées par des virgules. Les différents objets sont ensuite regroupés grâce à un séparateur `;`.

Le processus inverse repose sur un découpage de la chaîne selon les points-virgules, puis selon les virgules pour extraire les champs numériques. Chaque élément est converti, puis encapsulé dans une instance d'`ObjetSpatial`. Les erreurs de conversion numérique sont traitées afin d'assurer la stabilité du système.

Le réseau fournit des statistiques en temps réel, notamment le nombre total d'objets

enregistrés et une copie sécurisée du registre grâce à la méthode `getObjetsConnus()`. Des logs détaillés permettent de tracer précisément chaque opération de stockage et de mise à jour.

Chaque satellite communique via des sockets TCP, établissant ainsi un échange bidirectionnel fiable basé sur un schéma commande-réponse. Les données issues de plusieurs sources sont fusionnées afin de permettre une détection collaborative des obstacles.

La simulation exploite immédiatement les données reçues pour la mise à jour des trajectoires et l'évitement d'obstacles. L'arrêt du serveur est synchronisé avec l'arrêt de la simulation.

4.4 Antenne

La classe **Antenne** représente le module de communication de réseau d'un satellite. Elle établit une connexion TCP avec un serveur central, transmet les objets spatiaux ddétectés et reçoit la liste globales des objets connus afin de permettre la coordination entre satellites. L'envoie et la réception suivent un protocole texte simple basé sur des commendes tels que **DATA**, **GET OBSTACLES** et des réponses structurées **OK**, **ERR** La classe assure par ailleurs la sérialisation et dé-sérialisation des objets spatiaux dans un format compact, et garantit la sécurité de l'état de connexion grâce à un *AtomicBoolean*, un Boolean qui peut être lu et modifié sans risque de conflit lorsque plusieurs threads y accèdent en même temps.

4.5 Interface graphique

4.5.1 Control Panel

La classe ControlPanel constitue le tableau de bord principal de l'interface utilisateur de la simulation. Elle agit comme contrôleur permettant d'interagir avec l'environnemenet spatial, le **Space Panel** et de paraméter les entités physiques.

L'interface est conçue de manière dynamique via la la méthode *updateFieldsVisibility()*. Selon le type d'objet spatial sélectionné dans le menu déroulant, le panneau affiche ou masque contextuellement certains champs de saisie. On masque par exemple les paramètres orbitales pour les planètes.

Le coeur de la logique de la classe réside dans la métbode createObjetc. La classe gère par ailleurs l'établissement d'une connexion réseau.

Elle permet par ailleurs par les méthodes *saveSimulation* et *loadSimulation* de sauvegarder ou bien alors restaurer l'éat complet du système via un fichier binaire.

4.5.2 Space Panel

La classe **SpacePanel** est le coeur visuel et fonctionnel de la simultation. Elle permet le rendu graphique et la boucle de simlation physique en implémentant une évolution temporrelle du système. La méthode surchargée *paintComponent* est responsable de l'affichage en temps réel. Elle dessine les éléments couche par couche. Elle permet la gestion du

zoom et du déplacement de la caméra virtuelle pour permettre l'exploration de l'espace, l'affichage des orbites théoriques, des trajectoires et des zones de détection des caméras et le dessin des objets spatiaux avec une gestion des couleurs dynamiques ou personnalisés. La boucle de simulation physique permet toutes les 16ms grâce à la classe *Timer* qui vient de *javax.swing* et la méthode *actionPerformed* de calculer les forces, d'intégrer le mouvement la gestion de collisions et la communication réseau. La classe implémente par ailleurs plusieurs interfaces d'écoute pour offrir une navigation fluide : le zoom se fait pas la molette de la souris et le déplacement par un clic de la souris en "drangant". Par ailleurs le survol de la souris calcul et cnverit les ocordonnées écran en coordonnées réelles permettant de transmettre des positions précises au **Control Panel** lors d'un clic ou bien la destination pour un satellite.

5 Contributions

Le projet a été principalement réalisé par Sheradj Drissi Mouad et Zaki Salih Amr, qui ont assuré la modélisation du système (diagramme de classes, diagrammes de séquence) ainsi que l'ensemble du développement. La partie codage a été réalisée avec l'appui des IA Claude et DeepSeek, notamment pour la conception de l'interface graphique et l'optimisation du code. Les tâches ont été divisées pour que le travail final soit équitable. Sheradj Drissi Mouad a réalisé la modélisation du diagramme de classes, la conception des classes liées aux objets spatiaux, la gestion des collisions ainsi que de l'implémentation de la base de données avec ses mécanismes de sauvegarde et de chargement. Zaki Salih Amr a réalisé les diagrammes d'objets et de séquence, implémenté les classes Satellite, Réseau de satellites et Antenne, et développé le système de communication entre satellites, la gestion complète des trajectoires a été réalise par les deux membre du binôme. Il est important de souligner que le développement de l'interface graphique a été fortement appuyé par l'utilisation d'outils d'intelligence artificielle, notamment Claude et DeepSeek, qui ont facilité la conception visuelle. Malgré cet appui, la compréhension, l'intégration et l'adaptation de ces contributions automatisées ont été entièrement réalisées par les membres du binôme, garantissant la cohérence et la maîtrise technique du projet.

6 Analyse des réalisations

Lors du projet nous avons été confrontés à plusieurs obstacles qui ont troublé notre avancé. Par manque de temps, le modèle IA n'a pas pu être implémenté néanmoins nous avions une vision claire de la manière dont cela aurait pu être implémenté. L'idée initiale était de mettre en place un module d'apprentissage par renforcement, basé sur le Q-learning, permettant au satellite d'apprendre progressivement à adapter sa trajectoire en fonction de son environnement. Concrètement, l'IA aurait observé l'état du système (position, vitesse, distance aux obstacles, direction de la cible) et choisi parmi plusieurs actions possibles (avancer, tourner, accélérer, ralentir). Une fonction de récompense aurait encouragé les comportements efficaces (comme éviter une collision ou réduire la distance à la cible) et pénalisé les actions risquées ou inefficaces. Le modèle aurait affiné sa politique décisionnelle afin de naviguer de manière de plus en plus autonome, même en présence d'obstacles

dynamiques ou de perturbations imprévues. Par ailleurs pour la base de données, nous avions intégré une base de données à chacun des satellites, cependant nous étions confrontés au problème de taille de fichier. En effet, le satellite donnait la position, la taille, le vecteur vitesse, la masse etc à chaque tick des astres qu'il captait créant dès lors des fichiers devenant bien trop volumineux bien trop vite. Par ailleurs, pour la détection de satellite, celle-ci n'est pas très réaliste puisque l'on connaît les objets spatiaux à l'avance et qu'ils sont rajoutés dans sa liste d'astres détectés s'ils sont dans le champ de vision de sa caméra. Malgré tout cela, le projet a été mené à bien et les objectifs principaux qui sont de déterminer une trajectoire optimale pour un satellite dans un système planétaire est donc bien réalisé. Nous n'avons par ailleurs pas réalisé de système héliocentrique car cela aurait impliqué d'intégrer en permanence la force du Soleil dans les calculs d'orbite de chaque objet, en plus de la force de la planète autour de laquelle l'objet orbite. Cela impose un modèle de mécanique céleste beaucoup plus complexe, où chaque trajectoire résulte de l'interaction entre plusieurs corps massifs (modèle n-corps).

7 Conclusion

Pour conclure, nous avons apprécié de pouvoir travailler sur ce projet de simulation de trajectoire autonome d'un satellite. Bien que nous ne partions pas de zéro en Java, nous n'avions encore jamais mené un projet aussi complet intégrant à la fois des calculs physiques, de l'algorithmique (A^*) et de la communication réseau.

Ce fut donc l'occasion d'approfondir nos connaissances, de consolider nos bases et de mettre en pratique les concepts avancés de la programmation orientée objet (héritage, encapsulation, modularité) dans une problématique concrète inspirée du domaine spatial. Ce projet nous a permis de mieux comprendre la structuration d'un programme Java complexe, la gestion des interactions entre objets spatiaux (planètes, satellites, obstacles), et d'aborder la conception d'une interface graphique, pour laquelle nous avons su tirer parti d'outils d'IA comme Claude et DeepSeek.

Concernant le travail en groupe, nous pensons avoir su gérer efficacement le temps imparti en travaillant de manière régulière. Nous étions très motivés à l'idée de voir fonctionner ce que nous avions d'abord imaginé dans nos diagrammes de classes et de séquences. La répartition des tâches entre la modélisation physique et la base de données d'un côté, et le système de communication et de trajectoire de l'autre s'est faite naturellement. Malgré l'abandon du module d'IA par manque de temps, nous sommes fiers d'avoir atteint l'objectif principal : déterminer une trajectoire optimale tout en gérant un réseau de satellites interconnectés.

Bibliography

[1] *Mécanique : Fondements et applications*. Femto Physique. Disponible sur : https://femto-physique.fr/mecanique/pdf/book_meca.pdf

[2] Pérez, J.-P. *Mécanique : Fondements et applications*. Dunod. Disponible sur : https://perso.crans.org/sylvainrey/BiblioPhysique/Physique/MÃ¨canique/[PÃ¨rez]MÃ¨canique-Fondementsetapplications.pdf

[3] *Les lois de Kepler et les satellites*. Lycée Senghor. Disponible sur : https://senghor.lycee.ac-normandie.fr/IMG/pdf/microsoft_word_-_kepleretsatellites.pdf

[4] *Physique Nucléaire*. Université des Frères Mentouri Constantine. Disponible sur : https://fac.umc.edu.dz/fse/assets/img/PhysiqueNuclÃ¨aire.pdf

[5] *Collisions élastiques et inélastiques*. Collège de Maisonneuve, Département de Physique. Disponible sur : https://physique.cmaisonneuve.qc.ca/svezina/nya/note_nya/NYA_XXI_Chap1.12c.pdf

[6] *Collision inélastique*. Wikipédia, l'encyclopédie libre. Disponible sur : https://fr.wikipedia.org/wiki/Collision_inÃ¨lastique

[7] *Propulsion spatial*. Wikipédia, l'encyclopédie libre. Disponible sur : https://fr.wikipedia.org/wiki/Propulsion_spatiale